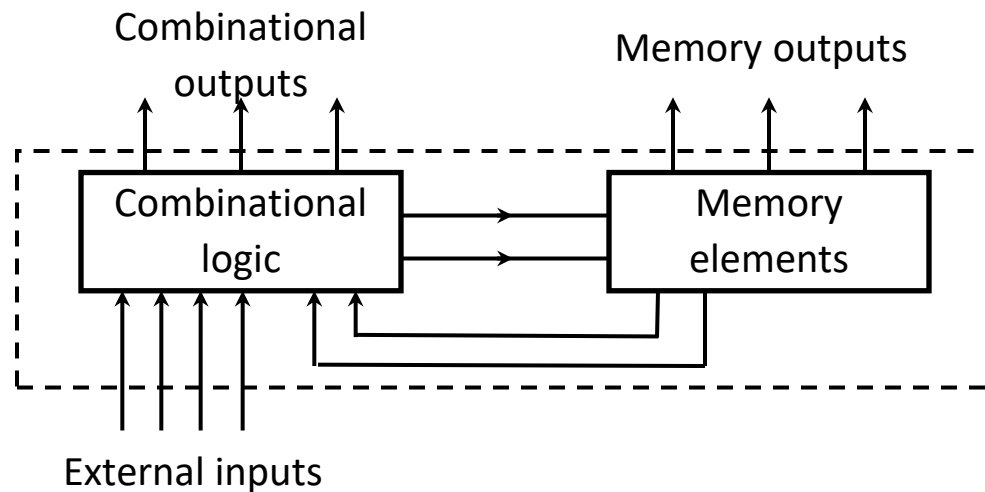


Sequential Circuits

- A **sequential circuit** consists of a *feedback path*, and employs some *memory elements*.



Sequential circuit = Combinational logic + Memory Elements

Sequential circuits are digital circuits in which the outputs depend not only on the current inputs, but also on the previous state of the output.

<u>Combinational Logic Circuits</u>	<u>Sequential Logic Circuits</u>
Output is a function of the present inputs (Time Independent Logic).	Output is a function of clock, present inputs and the previous states of the system.
Do not have the ability to store data (state).	Have memory to store the present states that is sent as control input (enable) for the next operation.
It does not require any feedback. It simply outputs the input according to the logic designed.	It involves feedback from output to input that is stored in the memory for the next operation.
Used mainly for Arithmetic and Boolean operations.	Used for storing data (and hence used in RAM).
Logic gates are the elementary building blocks.	Flip flops (binary storage device) are the elementary building unit.
Independent of clock and hence does not require triggering to operate.	Clocked (Triggered for operation with electronic pulses).
Example: Adder [$1+0=1$; Dependency only on present inputs i.e., 1 and 0].	Example: Counter [Previous O/P +1=Current O/P; Dependency on present input as well as previous state].

Introduction

- There are two types of sequential circuits:
 - ❖ *synchronous*: outputs change only at specific time
 - ❖ *asynchronous*: outputs change at any time
- *Multivibrator*: a class of sequential circuits. They can be:
 - ❖ *bistable* (2 stable states)
 - ❖ *monostable* or *one-shot* (1 stable state)
 - ❖ *astable* (no stable state)
- Bistable logic devices: *latches* and *flip-flops*.
- Latches and flip-flops differ in the method used for changing their state.

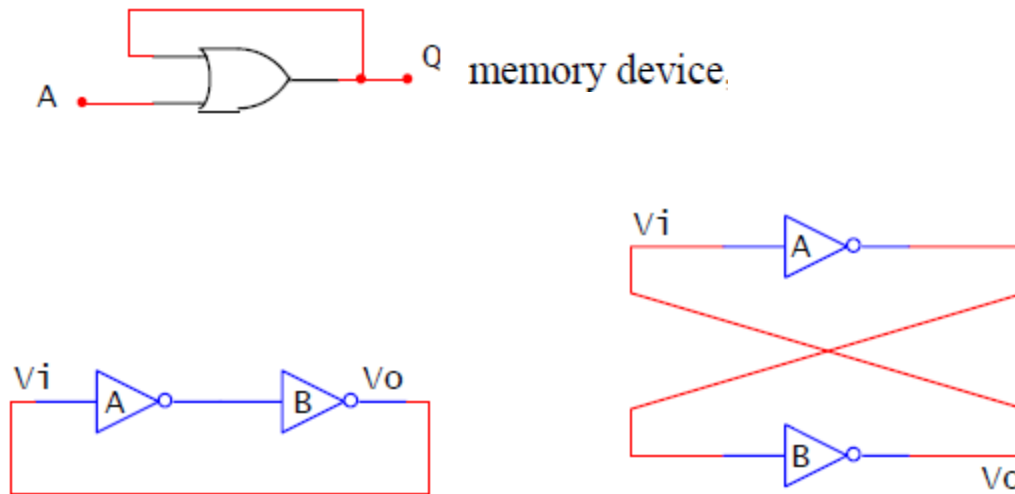
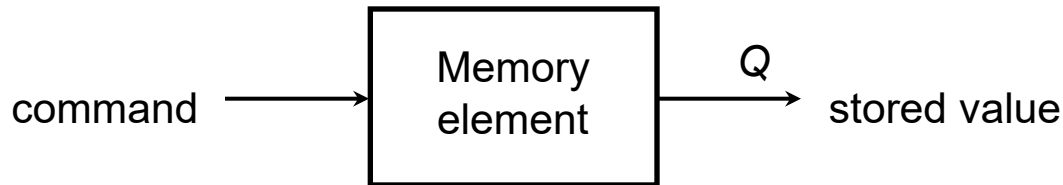


Figure 2. Fundamental latch

The circuit have two stable states, it is called bistable circuit.

Memory Elements

- **Memory element:** a device which can remember value indefinitely or change value on command from its inputs.



- **Characteristic table:**

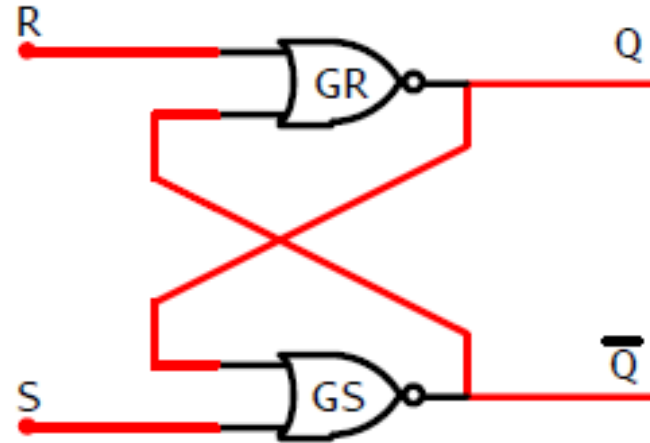
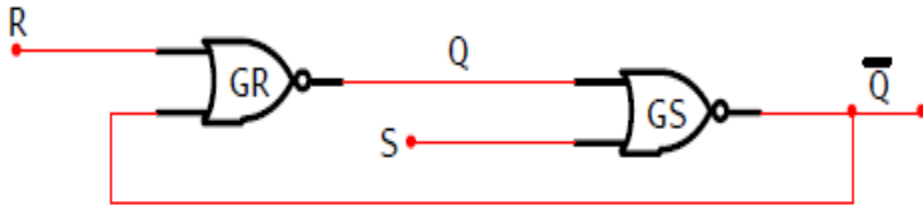
Command (at time t)	$Q(t)$	$Q(t+1)$
Set	X	1
Reset	X	0
Memorise / No Change	0	0
	1	1

$Q(t)$: current state

$Q(t+1)$ or Q^+ : next state

The Set-Reset (SR)

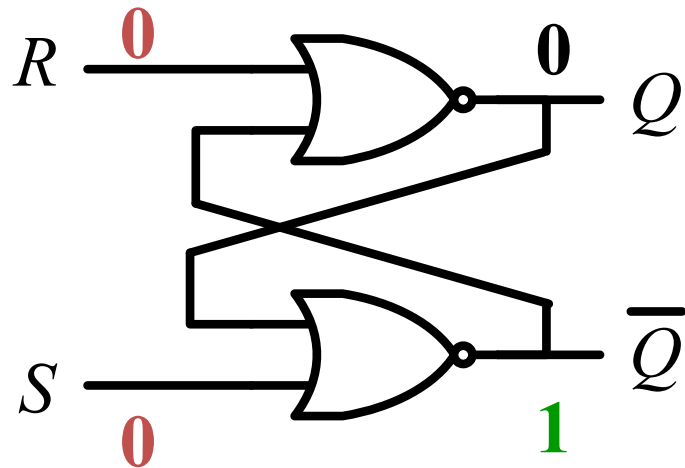
SR Latch:



Latch is a bistable memory device with level sensitive triggering (no clock). It is an asynchronous circuit.

Latches

- *SR* Latch



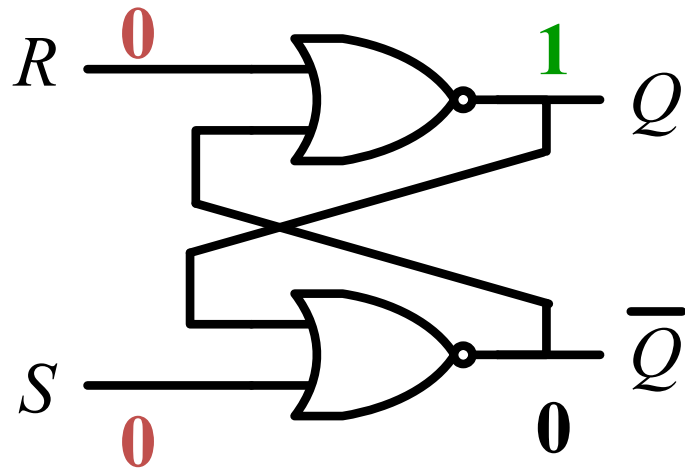
S	R	Q_0	Q	Q'
0	0	0	0	1

$Q = Q_0$

Initial Value

Latches

- *SR* Latch



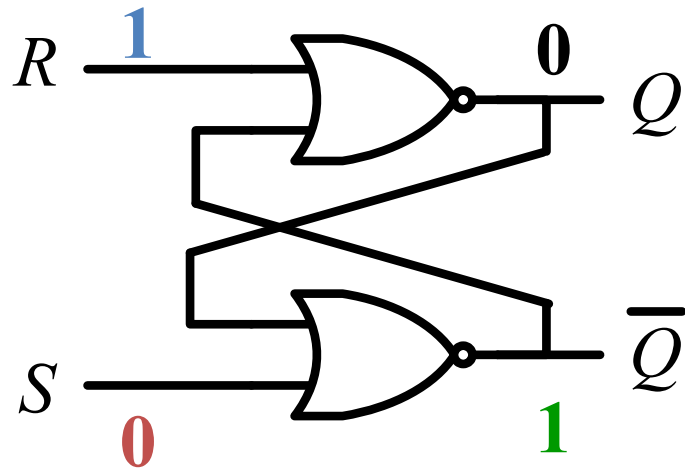
S	R	Q_0	Q	Q'
0	0	0	0	1
0	0	1	1	0

$Q = Q_0$

$Q = Q_0$

Latches

- *SR* Latch

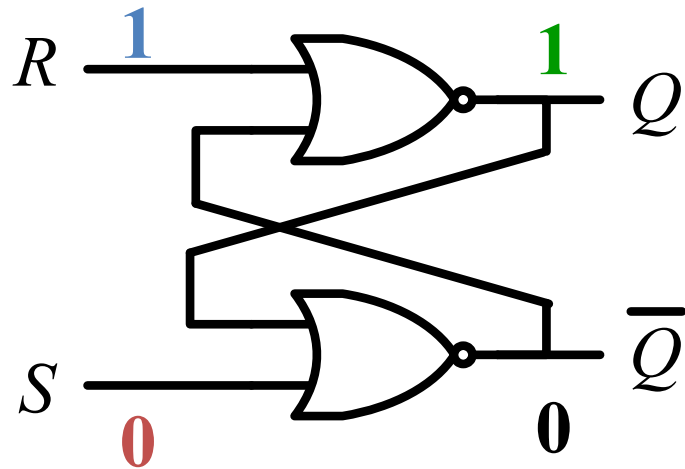


S	R	Q_0	Q	Q'
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1

} $Q = Q_0$
 $Q = 0$

Latches

- *SR* Latch



S	R	Q_0	Q	Q'
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	0	1

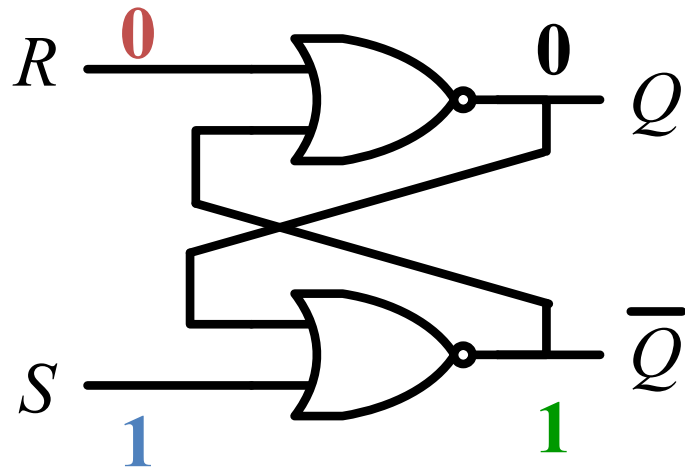
} $Q = Q_0$

$Q = 0$

$Q = 0$

Latches

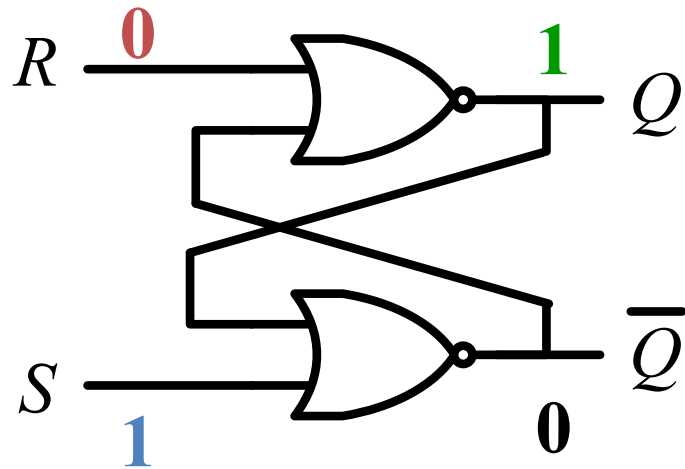
- *SR* Latch



S	R	Q_0	Q	Q'	
0	0	0	0	1	} $Q = Q_0$
0	0	1	1	0	
0	1	0	0	1	} $Q = 0$
0	1	1	0	1	
1	0	0	1	0	$Q = 1$

Latches

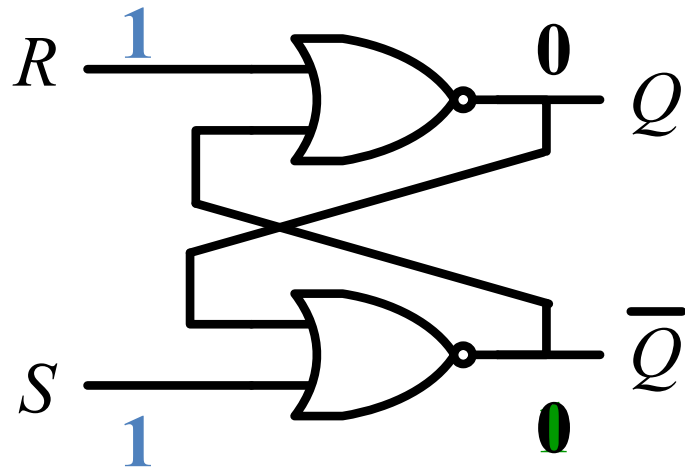
- *SR* Latch



S	R	Q_0	Q	Q'	
0	0	0	0	1	} $Q = Q_0$
0	0	1	1	0	
0	1	0	0	1	} $Q = 0$
0	1	1	0	1	
1	0	0	1	0	$Q = 1$
1	0	1	1	0	$Q = 1$

Latches

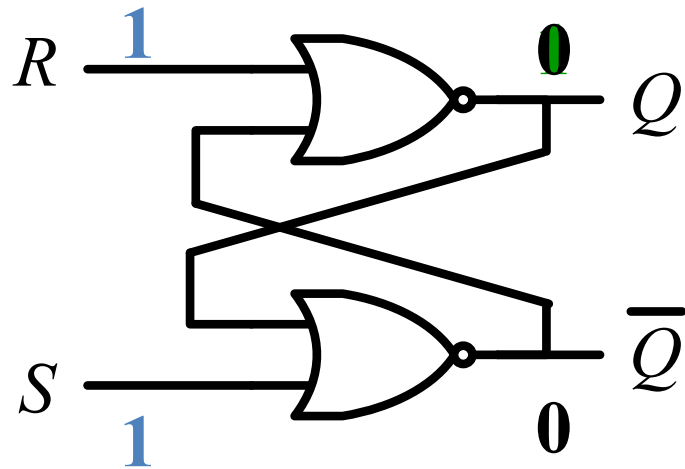
- *SR* Latch



S	R	Q_0	Q	Q'	
0	0	0	0	1	} $Q = Q_0$
0	0	1	1	0	
0	1	0	0	1	} $Q = 0$
0	1	1	0	1	
1	0	0	1	0	} $Q = 1$
1	0	1	1	0	
1	1	0	0	0	$Q = Q'$

Latches

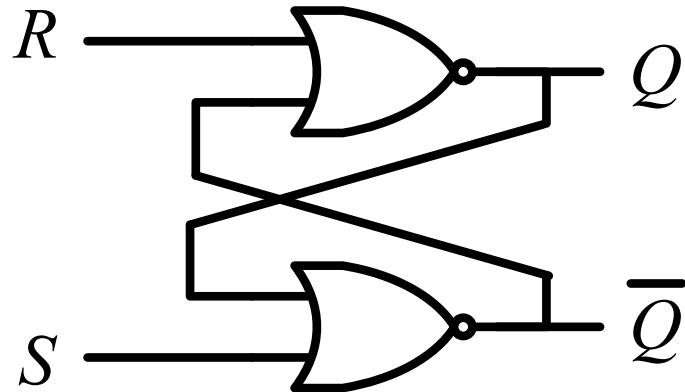
- *SR* Latch



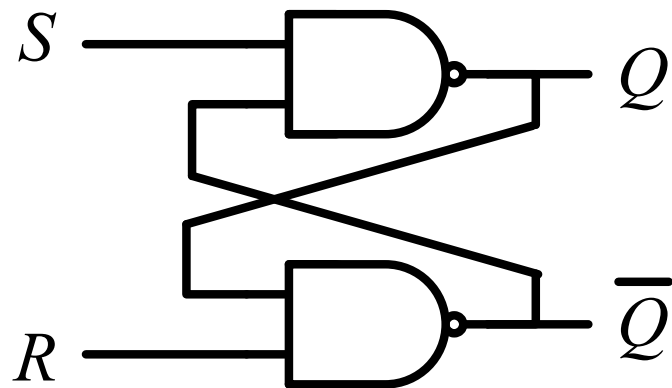
S	R	Q_0	Q	Q'	
0	0	0	0	1	} $Q = Q_0$
0	0	1	1	0	
0	1	0	0	1	} $Q = 0$
0	1	1	0	1	
1	0	0	1	0	} $Q = 1$
1	0	1	1	0	
1	1	0	0	0	$Q = Q'$
1	1	1	0	0	$Q = Q'$

Latches

- *SR* Latch



S	R	Q	
0	0	Q_0	No change
0	1	0	Reset
1	0	1	Set
1	1	$Q=Q'=0$	Invalid



S	R	Q	
0	0	$Q=Q'=1$	Invalid
0	1	1	Set
1	0	0	Reset
1	1	Q_0	No change

S-R Latch

- *Complementary* outputs: Q and Q' .
- When Q is HIGH, the latch is in *SET* state.
- When Q is LOW, the latch is in *RESET* state.
- For *active-HIGH input S-R latch* (also known as NOR gate latch),
 - $R=\text{HIGH}$ (and $S=\text{LOW}$) $\Rightarrow$ RESET state
 - $S=\text{HIGH}$ (and $R=\text{LOW}$) $\Rightarrow$ SET state
 - both inputs LOW $\Rightarrow$ no change
 - both inputs HIGH $\Rightarrow$ Q and Q' both LOW (invalid)!

S-R Latch

- For *active-LOW input S'-R' latch* (also known as NAND gate latch),
 - $R' = \text{LOW}$ (and $S' = \text{HIGH}$) $\Rightarrow$ RESET state
 - $S' = \text{LOW}$ (and $R' = \text{HIGH}$) $\Rightarrow$ SET state
 - both inputs HIGH $\Rightarrow$ no change
 - both inputs LOW $\Rightarrow$ Q and Q' both HIGH (invalid)!
- Drawback of S-R latch: invalid condition exists and must be avoided.

Controlled Latches

- NAND Based SR Latch with Control Input*

En	S	R	Q
0	x	x	Q_0
1	0	0	Q_0
1	0	1	0
1	1	0	1
1	1	1	$Q=Q'$

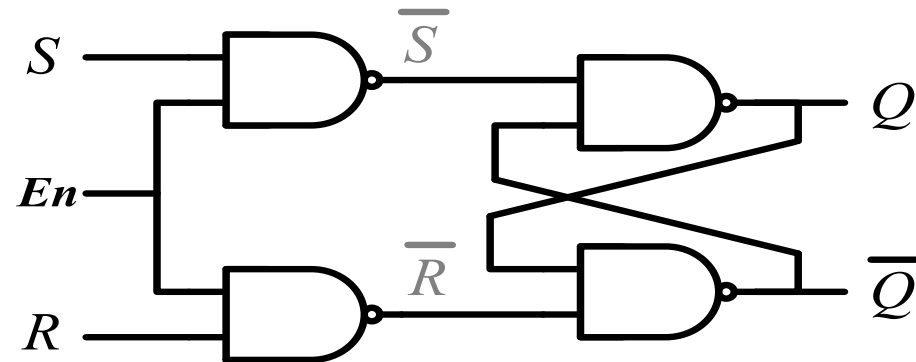
No change

No change

Reset

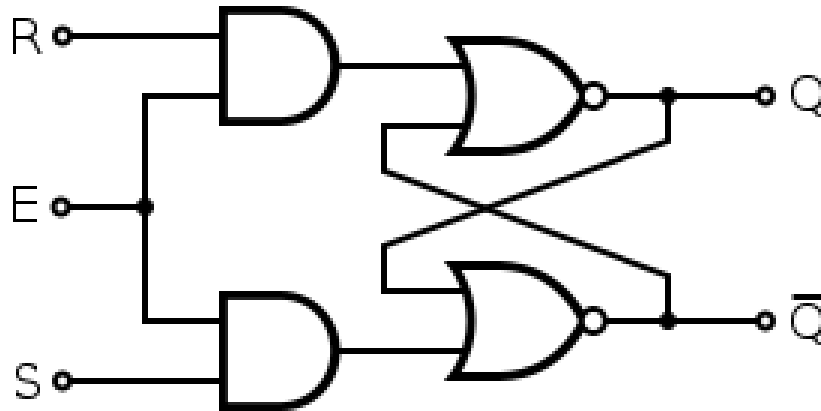
Set

Invalid



Controlled Latches

- NOR Based SR Latch with Control Input*



<i>E</i>	<i>S</i>	<i>R</i>	<i>Q</i>
0	x	x	Q_0
1	0	0	Q_0
1	0	1	0
1	1	0	1
1	1	1	$Q=Q'$

No change

No change

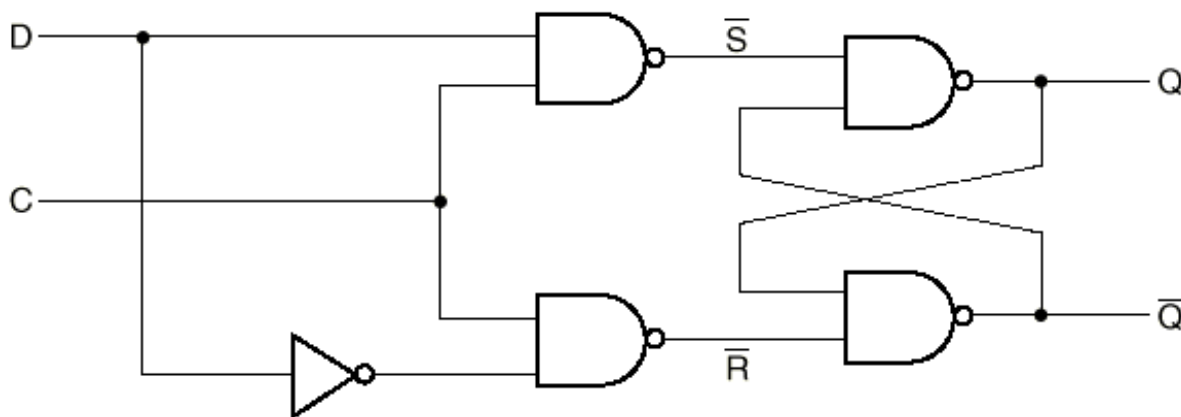
Reset

Set

Invalid

D Latch

- One way to eliminate the undesirable indeterminate state in the RS flip flop is to ensure that inputs S and R are never 1 simultaneously. This is done in the *D latch*:



C	D	Next state of Q
0	X	No change
1	0	Q = 0; Reset state
1	1	Q = 1; Set state

Gated D Latch

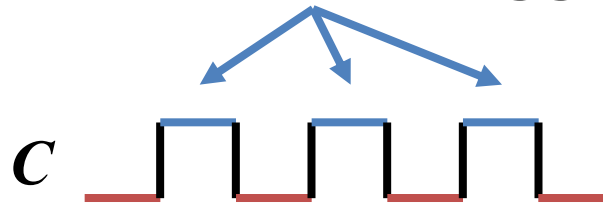
- When EN is HIGH,
 - ❖ $D=HIGH \rightarrow$ latch is SET
 - ❖ $D=LOW \rightarrow$ latch is RESET
- Hence when EN is HIGH, Q 'follows' the D (data) input.
- Characteristic table:

EN	D	$Q(t+1)$	
1	0	0	Reset
1	1	1	Set
0	X	$Q(t)$	No change

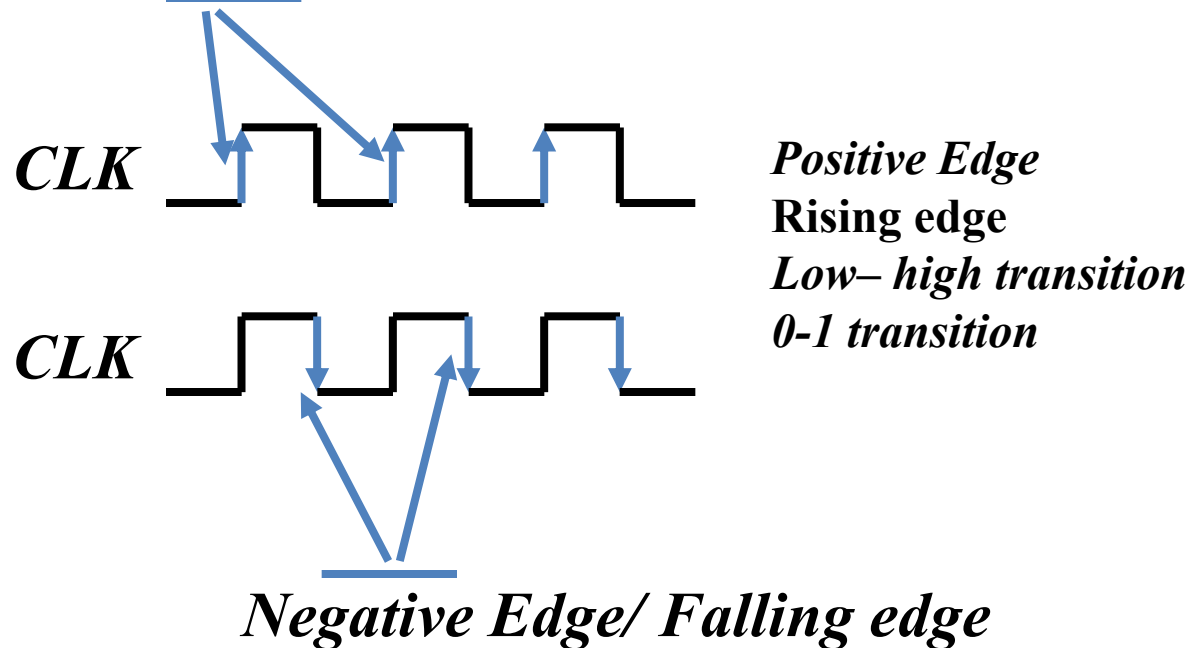
When $EN=1$, $Q(t+1) = D$

Flip-Flops

- Controlled latches are level-triggered



- Flip-Flops are edge-triggered

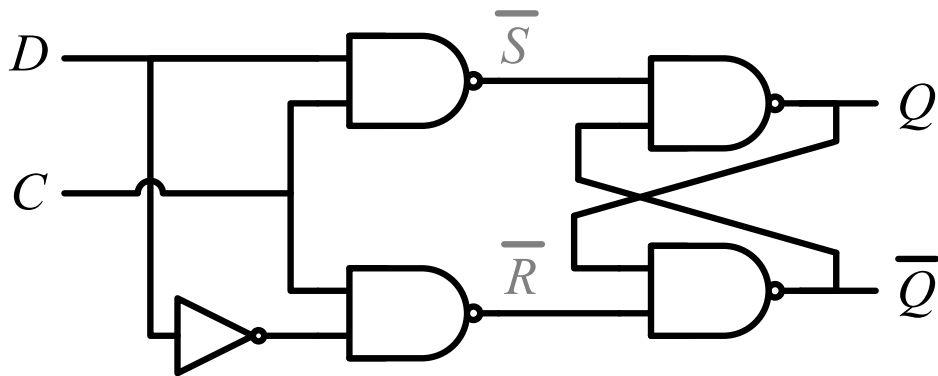


Memory Elements

- Two types of triggering/activation/sensitive:
 - ❖ pulse-triggered/level triggering
 - ❖ edge-triggered
- Pulse-triggered/level triggering(Latches)
 - ◆ High-level sensitive
 - ◆ Low-level sensitive
- Edge-triggered (Flip-flops)
 - ◆ Rising (positive) edge triggered(ON = from 0 to 1; OFF = other time)
 - ◆ Falling (negative) edge triggered (ON = from 1 to 0; OFF = other time)
 - ◆ Dual-edge triggered

Controlled Latches

- D Latch ($D = \text{Data}$)



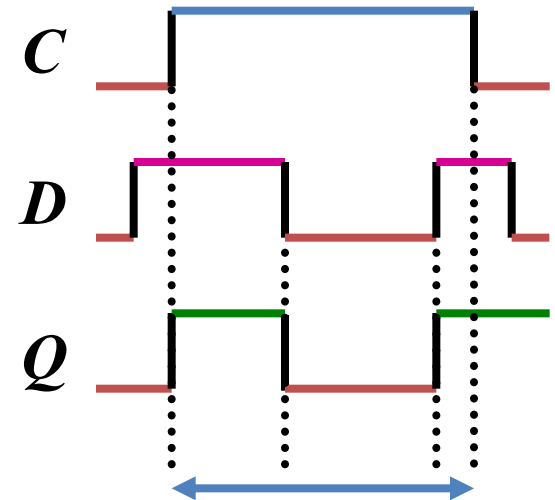
C	D	Q
0	x	Q_0
1	0	0
1	1	1

No change

Reset

Set

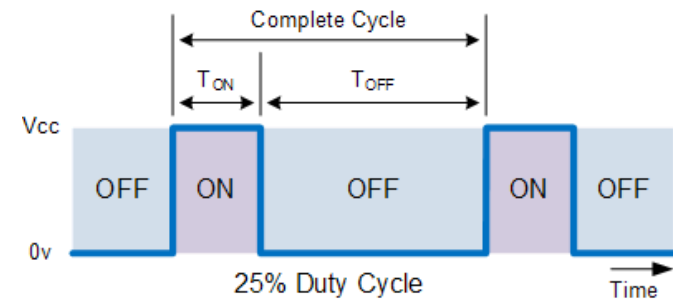
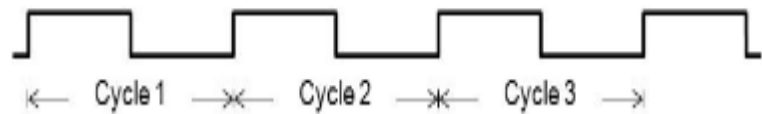
Timing Diagram



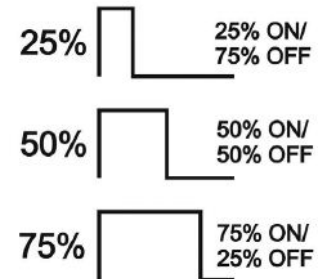
Output may change

CLOCK SIGNAL

- The clock signal is generally a rectangular pulse train.
- The clock signal is a periodic square wave that indefinitely rectangular switches values from 0 to 1 and 1 to 0 at fixed interval.
- Clock period = clock cycle (time)
- Clock Frequency = $1 / \text{clock period}$



- Duty cycle = $T_{\text{high}} / T_{\text{cycle}}$



Flip-flop	Latch
A flip-flop samples the inputs only at a clock event (rising edge, etc.)	A Latch samples the inputs continuously <i>whenever it is enabled</i> , that is, only when the enable signal is on. (or otherwise, it would be a wire, not a latch).
Flip-Flop are edge sensitive.	Latches are level sensitive.
Flipflop is sensitive to signal change and not on level. They can transfer data only at the single instant and data cannot be changed until next signal change.	Latch is sensitive to duration of pulse and can send or receive the data when the switch is on.
A flip-flop continuously checks its inputs and correspondingly changes its output only at times determined by clocking signal.	Latch is a device which continuously checks all its input and correspondingly changes its output, independent of the time determined by clocking signal.
It work's on the basis of clock pulses.	It is based on enable function input
It is a edge triggerred , it mean that the output and the next state input changes when there is a change in clock pulse whether it may a +ve or -ve clock pulse.	It is a level triggerred , it mean that the output of present state and input of the next state depends on the level that is binary input 1 or 0.

Latch Circuits: Not Suitable

- Latch circuits are not suitable in synchronous logic circuits.
- When the enable signal is active, the excitation inputs are gated directly to the output Q. Thus, any change in the excitation input immediately causes a change in the latch output.
- The problem is solved by using a special timing control signal called a *clock* to restrict the times at which the states of the memory elements may change.
- This leads us to the edge-triggered memory elements called *flip-flops*.

Flip Flop vs. Latch

Latches and flip-flops (FFs) are the basic building blocks of sequential circuits.

Latch:

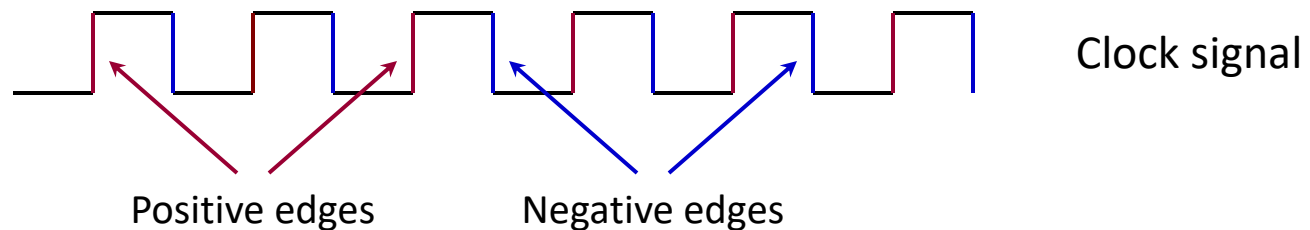
- Bistable memory device with level sensitive triggering (no clock), **all of its inputs continuously and changes its outputs, independent of a clocking signal.**
- Latch is based on Enable function inputs.
- Asynchronous Circuit.

Flip-flop:

- Bistable memory device with edge-triggering (with clock), **samples its inputs, and changes its output only at times determined by a clocking signal.**
- It works on basis of clock pulse.

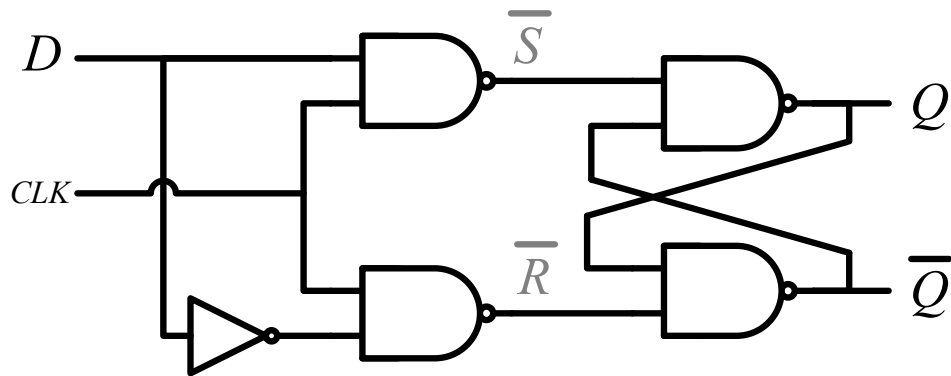
Edge-Triggered Flip-flops

- *Flip-flops*: synchronous bistable devices
- Output changes state at a specified point on a triggering input called the *clock*.
- Change state either at the *positive edge* (rising edge) or at the *negative edge* (falling edge) of the clock signal.



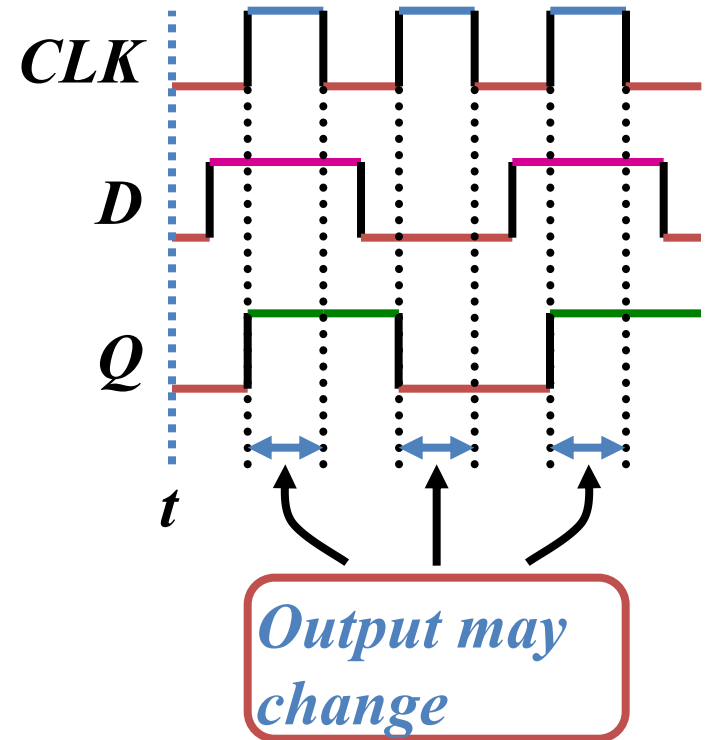
D Flip-Flop

- *D* Flip-flop



<i>CLK</i>	<i>D</i>	<i>Q</i>	
0	x	Q_0	No change
1	0	0	Reset
1	1	1	Set

Timing Diagram



SR Flip-flop

- NAND Based SR Flip-flop*

CLK	S	R	Q
0	x	x	Q_0
1	0	0	Q_0
1	0	1	0
1	1	0	1
1	1	1	$Q=Q'$

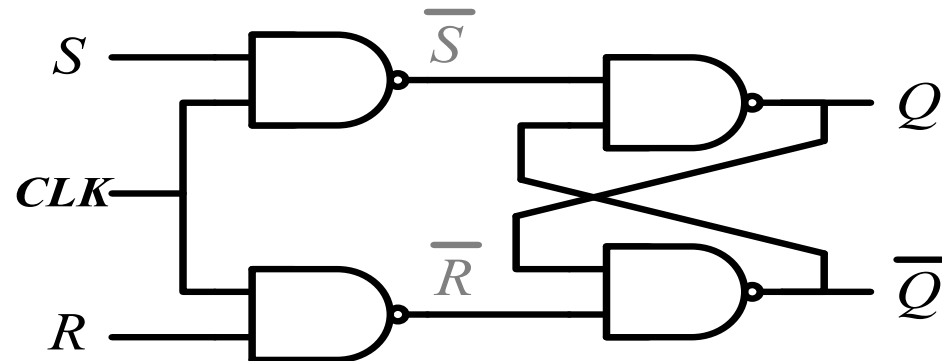
No change

No change

Reset

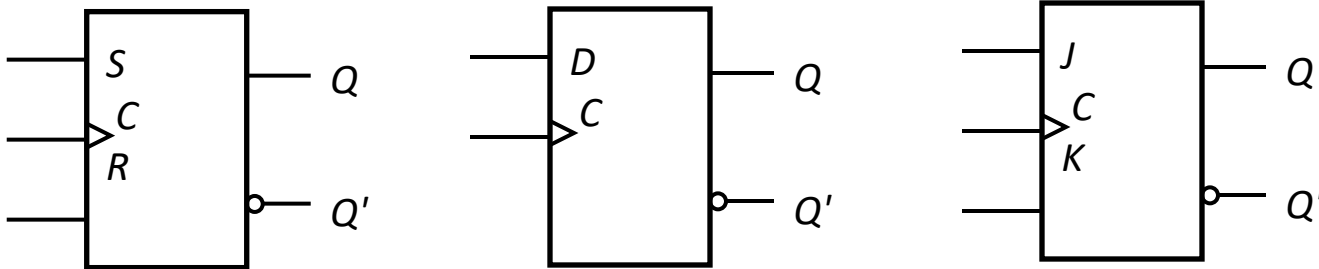
Set

Invalid

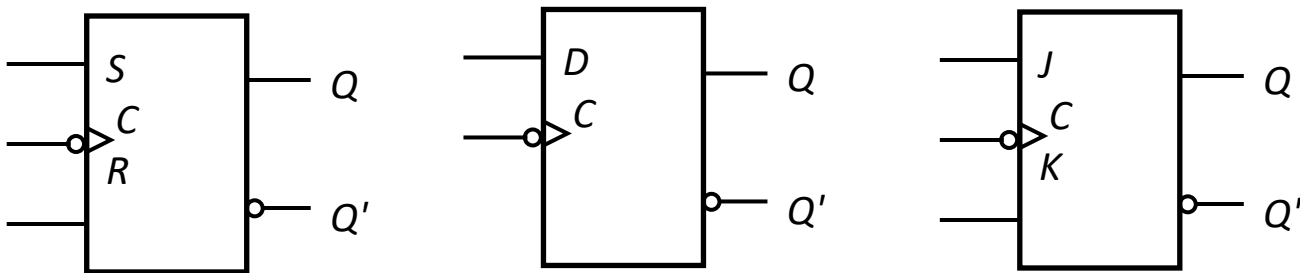


Edge-Triggered Flip-flops

- S-R, D and J-K edge-triggered flip-flops. Note the “>” symbol at the clock input.



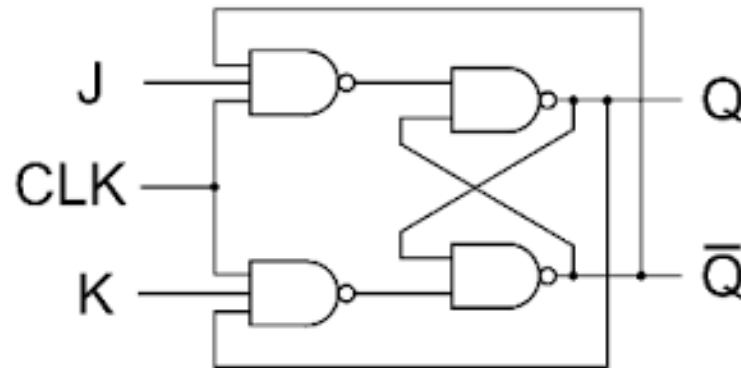
Positive edge-triggered flip-flops



Negative edge-triggered flip-flops

J-K Flip-flop

- NAND Based J-K flip-flop



- Characteristic table.

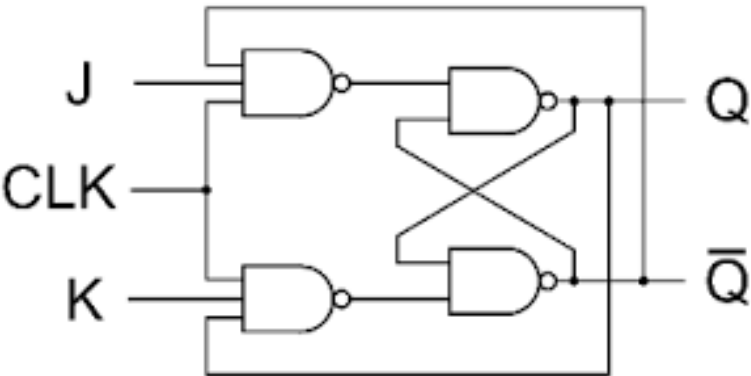
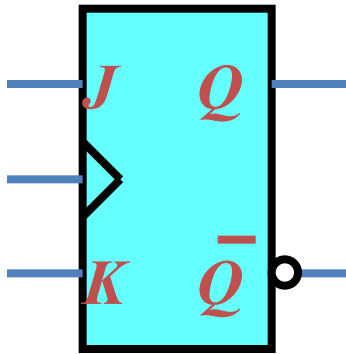
<i>J</i>	<i>K</i>	<i>CLK</i>	$Q(t+1)$	Comments
0	0	↑	$Q(t)$	No change
0	1	↑	0	Reset
1	0	↑	1	Set
1	1	↑	$Q(t)'$	Toggle

$$Q(t+1) = J.Q' + K'.Q$$

<i>Q</i>	<i>J</i>	<i>K</i>	$Q(t+1)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Flip-Flop Characteristic Equations

- Analysis / Derivation

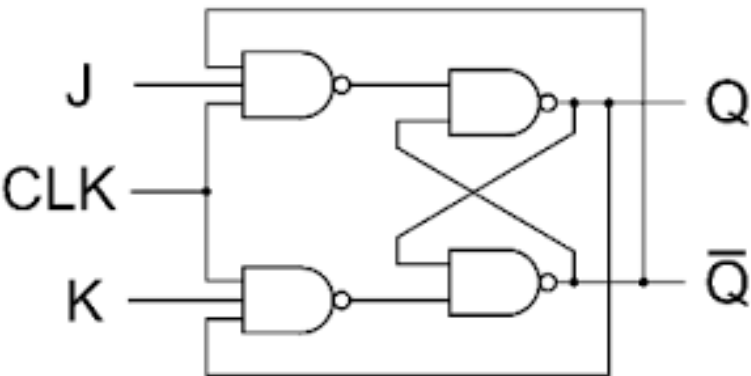
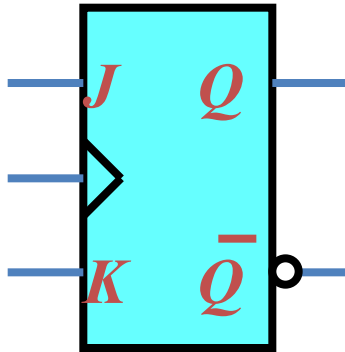


J	K	$Q(t)$	$Q(t+1)$
0	0	0	0
0	0	1	1
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

} No change

Flip-Flop Characteristic Equations

- Analysis / Derivation



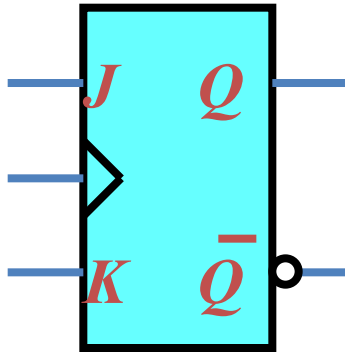
J	K	$Q(t)$	$Q(t+1)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	
1	0	1	
1	1	0	
1	1	1	

} No change

} Reset

Flip-Flop Characteristic Equations

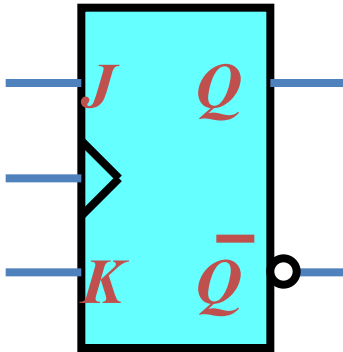
- Analysis / Derivation



J	K	$Q(t)$	$Q(t+1)$	
0	0	0	0	} No change
0	0	1	1	
0	1	0	0	} Reset
0	1	1	0	
1	0	0	1	} Set
1	0	1	1	
1	1	0		
1	1	1		

Flip-Flop Characteristic Equations

- Analysis /Derivation

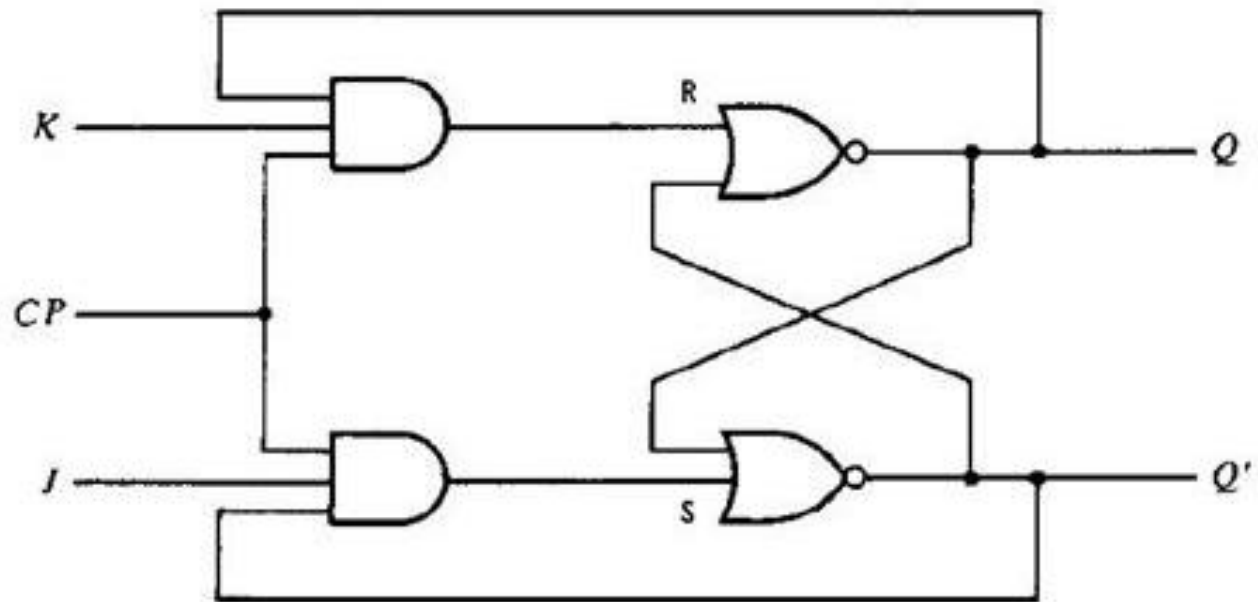


J	K	$Q(t)$	$Q(t+1)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

		K	
		0	1
J	0	0	1
	1	1	0
		Q	

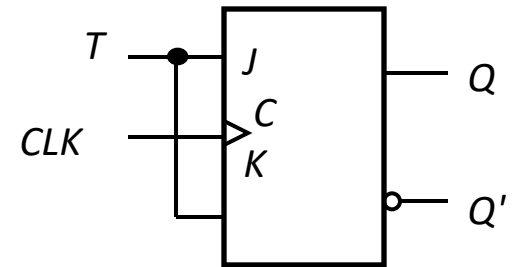
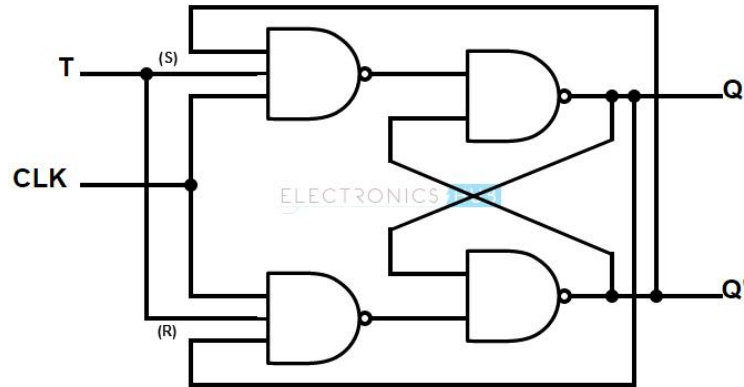
$$Q(t+1) = JQ' + K'Q$$

NOR Based J-K flip-flop



T Flip-flop

- **NAND Based T flip-flop**: single-input version of the J-K flip flop, formed by tying both inputs together.



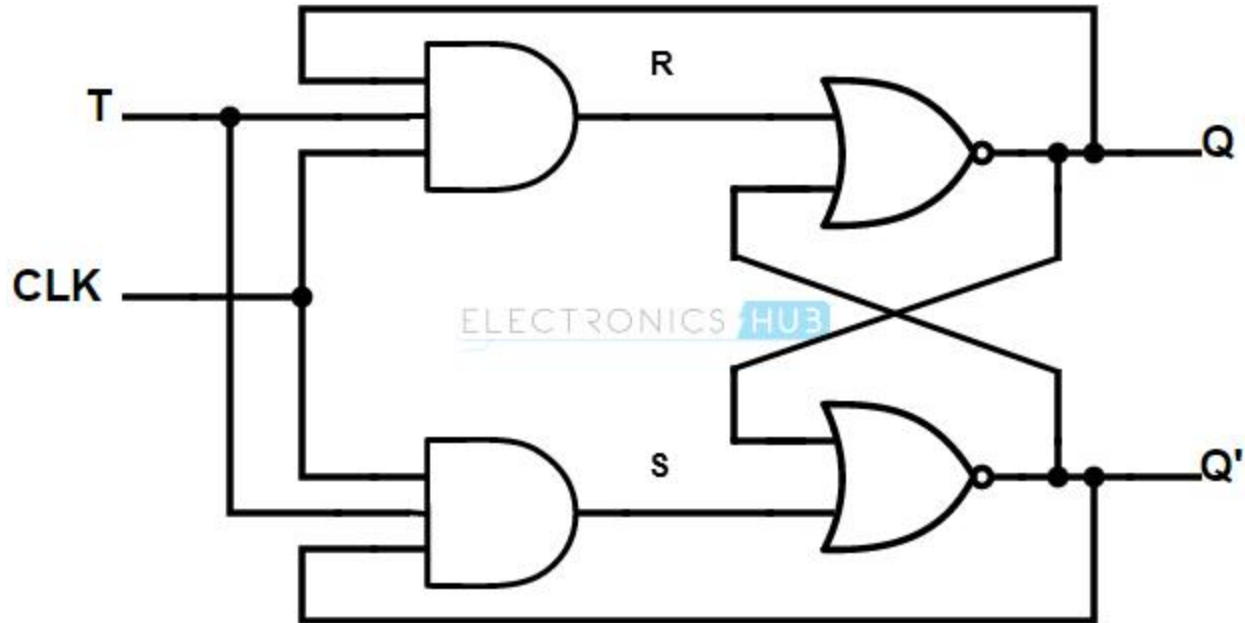
- Characteristic table.

T	CLK	$Q(t+1)$	Comments
0	↑	$Q(t)$	No change
1	↑	$Q(t)'$	Toggle

$$Q(t+1) = T.Q' + T'.Q$$

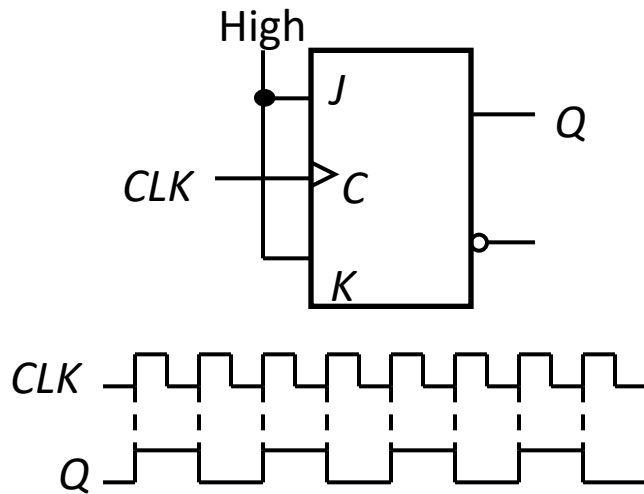
Q	T	$Q(t+1)$
0	0	0
0	1	1
1	0	1
1	1	0

NOR Based T flip-flop

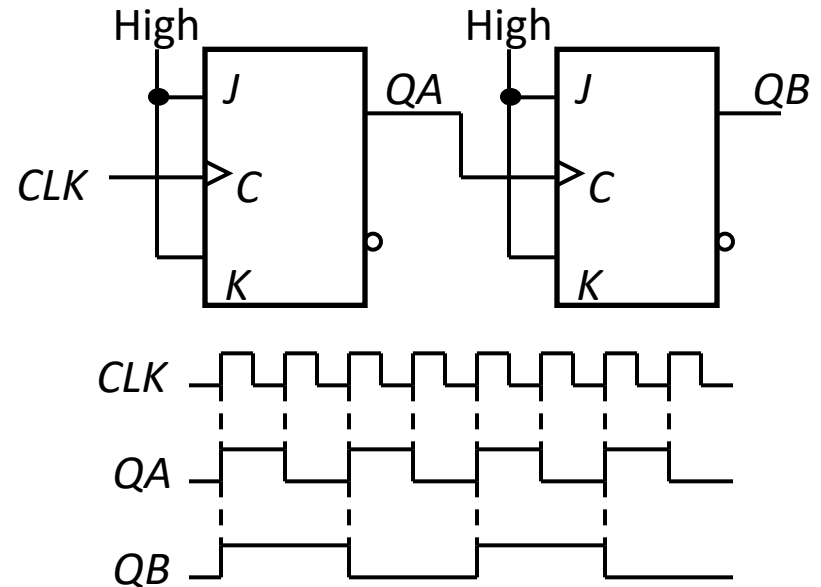


T Flip-flop

- Application: *Frequency division.*



Divide clock frequency by 2.



Divide clock frequency by 4.

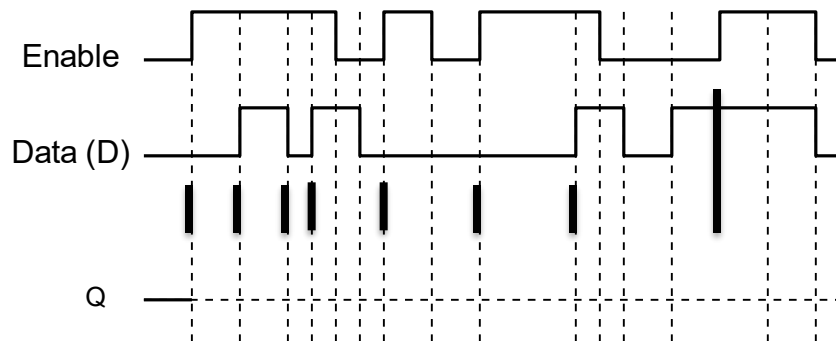
Complete the timing diagrams for :

(a) D Latch

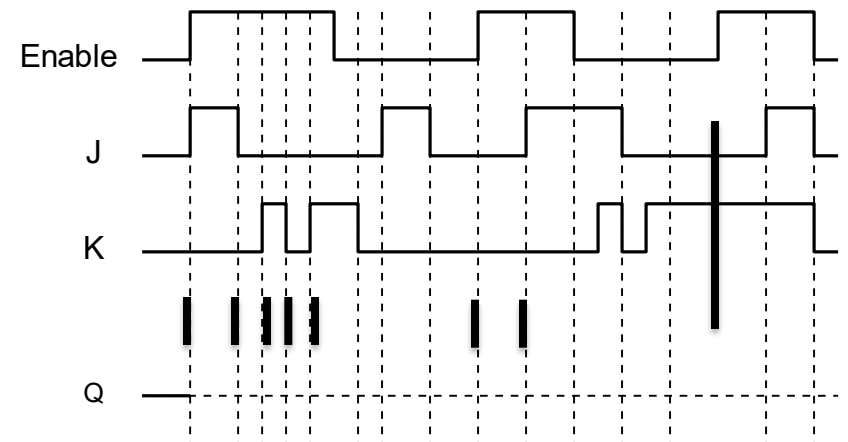
(b) JK Latch

Assume that for both cases the Q output is initially at logic zero.

(a)

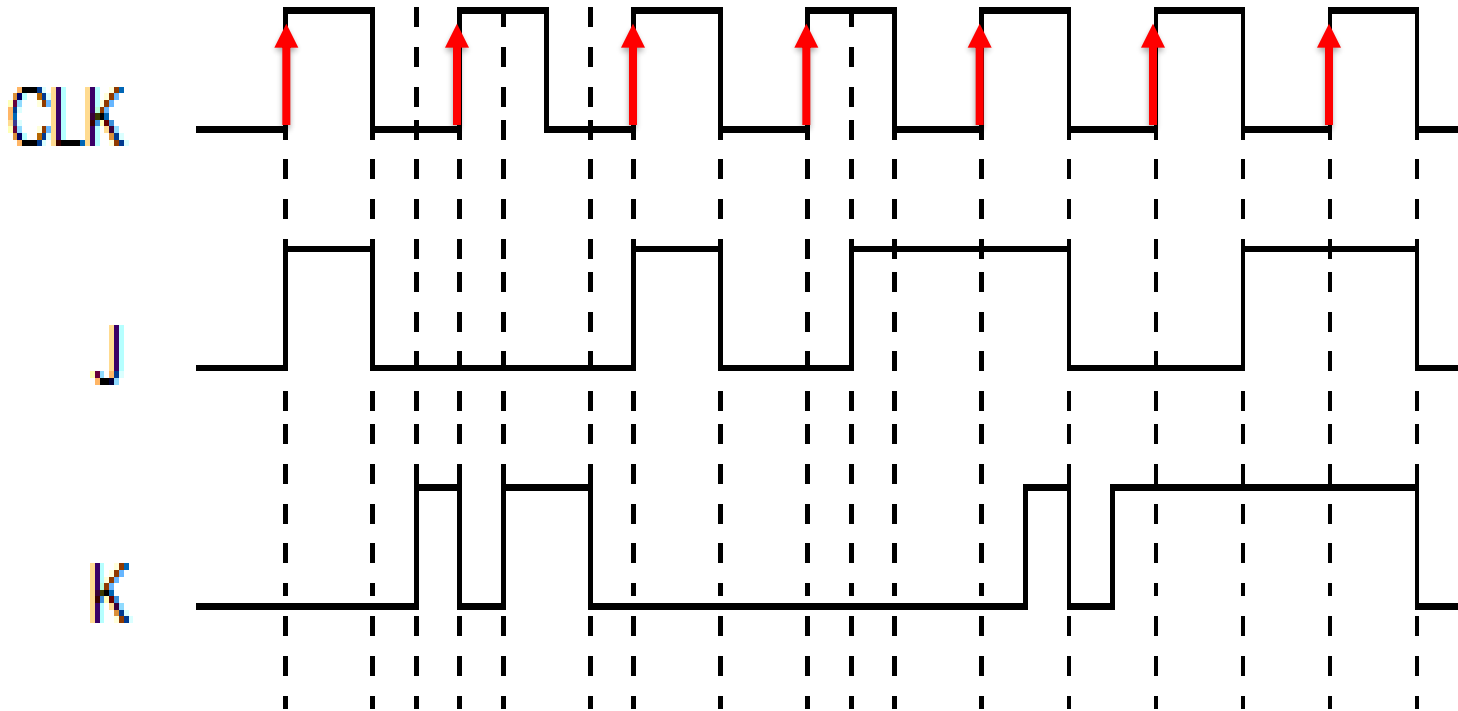


(b)

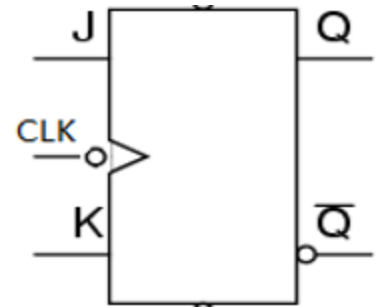
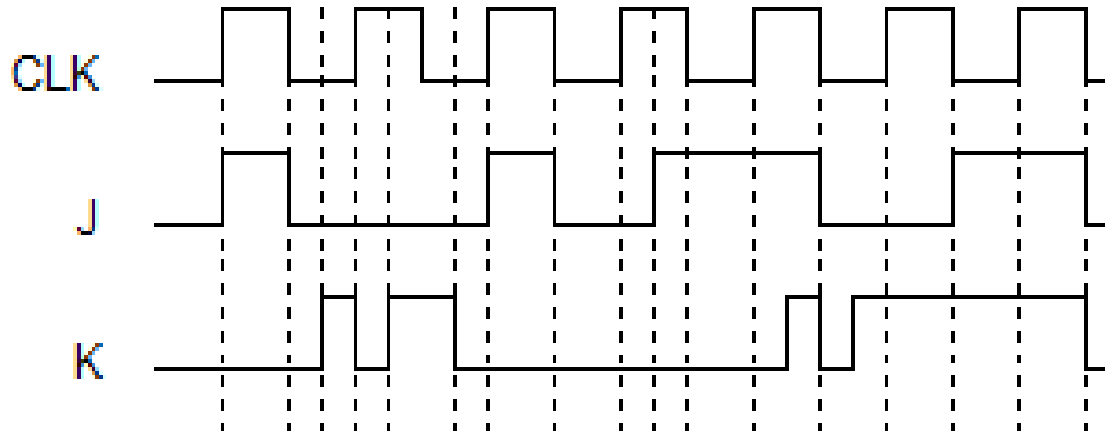


Transition points

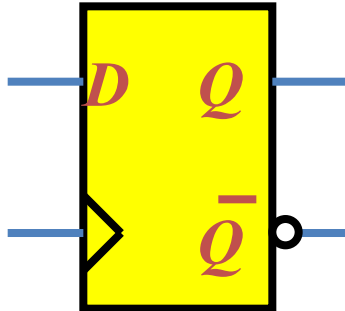
Complete the timing diagram for JK flip-flop.



Complete the timing diagram.



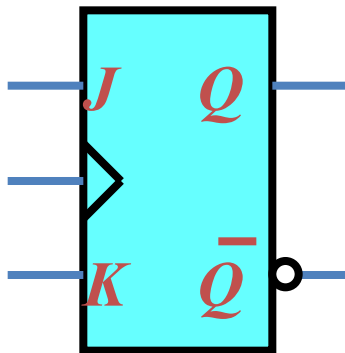
Flip-Flop Characteristic Tables



D	$Q(t+1)$
0	0
1	1

Reset

Set



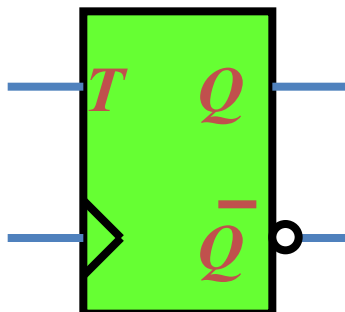
J	K	$Q(t+1)$
0	0	$Q(t)$
0	1	0
1	0	1
1	1	$Q'(t)$

No change

Reset

Set

Toggle

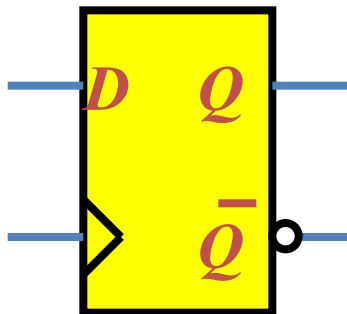


T	$Q(t+1)$
0	$Q(t)$
1	$Q'(t)$

No change

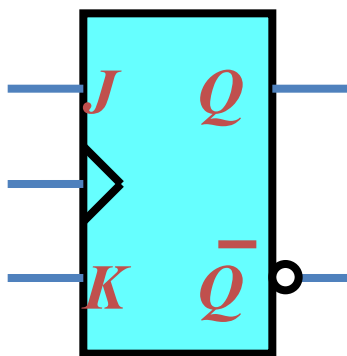
Toggle

Flip-Flop Characteristic Equations



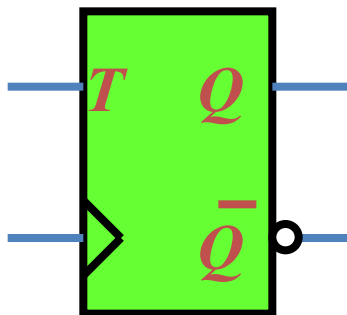
D	$Q(t+1)$
0	0
1	1

$$Q(t+1) = D$$



J	K	$Q(t+1)$
0	0	$Q(t)$
0	1	0
1	0	1
1	1	$Q'(t)$

$$Q(t+1) = JQ' + K'Q$$



T	$Q(t+1)$
0	$Q(t)$
1	$Q'(t)$

$$Q(t+1) = T \oplus Q$$

SR Flip-Flop

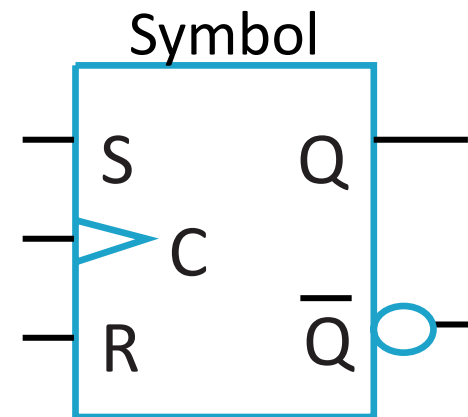
- **Characteristic Table**

S	R	$Q(t + 1)$	Operation
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	?	Undefined

Excitation table indicates the excitation or moves required to take the flip-flop from the present state to the next state.

- **Excitation Table**

$Q(t)$	$Q(t+ 1)$	S	R	Operation
0	0	0	X	No change
0	1	1	0	Set
1	0	0	1	Reset
1	1	X	0	No change



Q_0	Q	S	R	Operation
0	0	0	X	No change
0	1	1	0	Set
1	0	0	1	Reset
1	1	X	0	No change

S	R	Q_0	Q	Q'
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	0	1
1	0	0	1	0
1	0	1	1	0
1	1	0	-	-
1	1	1	-	-

JK Flip-Flop

- Characteristic Table

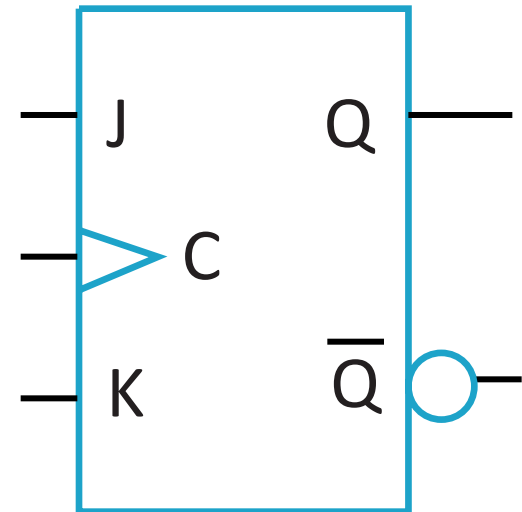
J	K	Q(t+1)	Operation
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	$\bar{Q}(t)$	Complement

- Excitation Table

Q(t)	Q(t + 1)	J	K	Operation
0	0	0	X	No change
0	1	1	X	Set
1	0	X	1	Reset
1	1	X	0	No Change

- Characteristic Equation
$$Q(t+1) = J \bar{Q}(t) + \bar{K} Q(t)$$

Symbol



Universal Programmable Flip Flop

T Flip-Flop

- Characteristic Table

T	Q(t+1)	Operation
0	$Q(t)$	No change
1	$\bar{Q}(t)$	Complement

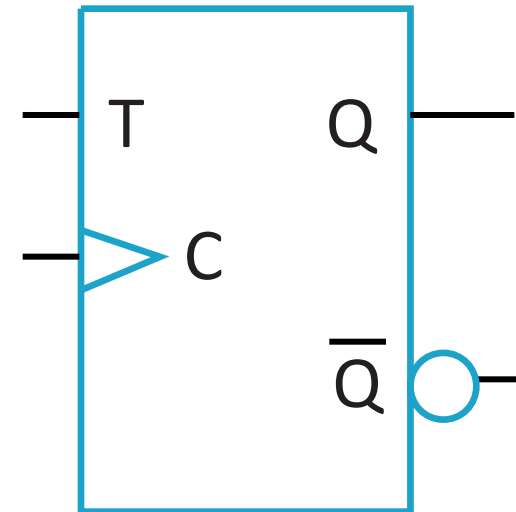
- Characteristic Equation

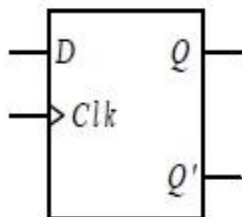
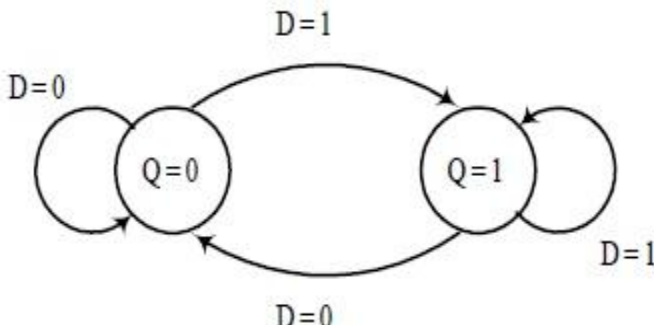
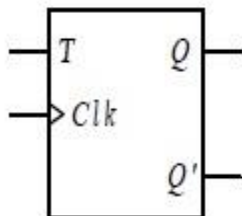
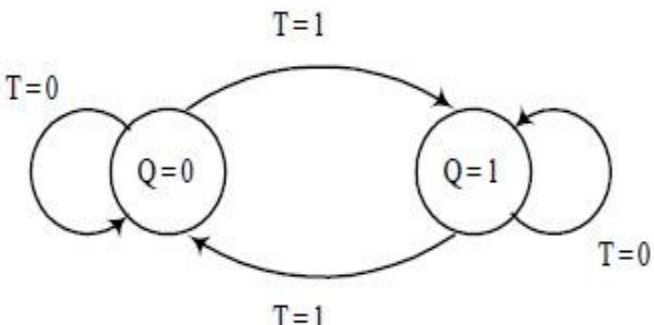
$$Q(t+1) = T \oplus Q(t)$$

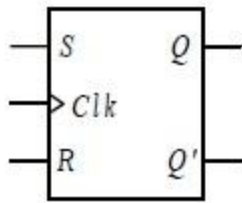
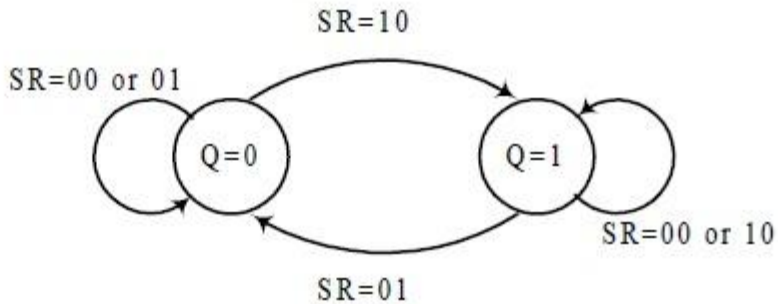
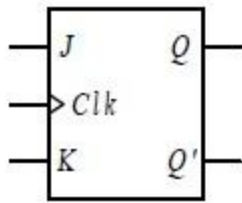
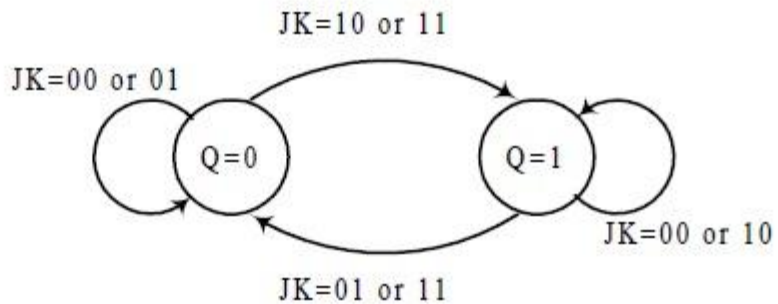
- Excitation Table

Q(t)	Q(t + 1)	T	Operation
0	0	0	No change
0	1	1	Complement
1	0	1	Complement
1	1	0	No Change

Symbol



Name / Symbol	Characteristic (Truth) Table	State Diagram / Characteristic Equations	Excitation Table																														
D 	<table><tr><th>D</th><th>Q</th><th>Q_{next}</th></tr><tr><td>0</td><td>×</td><td>0</td></tr><tr><td>1</td><td>×</td><td>1</td></tr></table>	D	Q	Q _{next}	0	×	0	1	×	1	 $Q_{next} = D$	<table><tr><th>Q</th><th>Q_{next}</th><th>D</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Q	Q _{next}	D	0	0	0	0	1	1	1	0	0	1	1	1						
D	Q	Q _{next}																															
0	×	0																															
1	×	1																															
Q	Q _{next}	D																															
0	0	0																															
0	1	1																															
1	0	0																															
1	1	1																															
T 	<table><tr><th>T</th><th>Q</th><th>Q_{next}</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	T	Q	Q _{next}	0	0	0	0	1	1	1	0	1	1	1	0	 $Q_{next} = TQ' + T'Q = T \oplus Q$	<table><tr><th>Q</th><th>Q_{next}</th><th>T</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	Q	Q _{next}	T	0	0	0	0	1	1	1	0	1	1	1	0
T	Q	Q _{next}																															
0	0	0																															
0	1	1																															
1	0	1																															
1	1	0																															
Q	Q _{next}	T																															
0	0	0																															
0	1	1																															
1	0	1																															
1	1	0																															

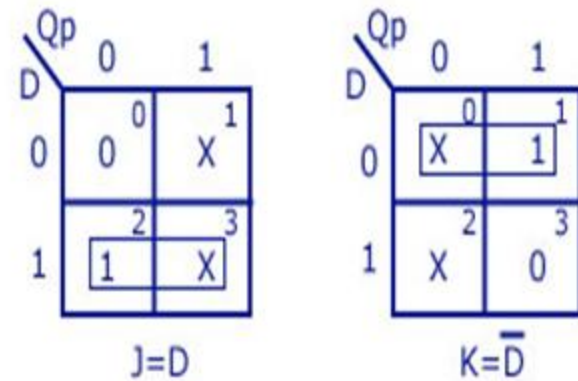
Name / Symbol	Characteristic (Truth) Table	State Diagram / Characteristic Equations	Excitation Table																																																								
SR 	<table> <tr> <th>S</th><th>R</th><th>Q</th><th>Q_{next}</th></tr> <tr><td>0</td><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>1</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td><td>×</td></tr> <tr><td>1</td><td>1</td><td>1</td><td>×</td></tr> </table>	S	R	Q	Q_{next}	0	0	0	0	0	0	1	1	0	1	0	0	0	1	1	0	1	0	0	1	1	0	1	1	1	1	0	×	1	1	1	×	 $Q_{next} = S + R'Q$ $SR = 0$	<table> <tr> <th>Q</th><th>Q_{next}</th><th>S</th><th>R</th></tr> <tr><td>0</td><td>0</td><td>0</td><td>×</td></tr> <tr><td>0</td><td>1</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>×</td><td>0</td></tr> </table>	Q	Q_{next}	S	R	0	0	0	×	0	1	1	0	1	0	0	1	1	1	×	0
S	R	Q	Q_{next}																																																								
0	0	0	0																																																								
0	0	1	1																																																								
0	1	0	0																																																								
0	1	1	0																																																								
1	0	0	1																																																								
1	0	1	1																																																								
1	1	0	×																																																								
1	1	1	×																																																								
Q	Q_{next}	S	R																																																								
0	0	0	×																																																								
0	1	1	0																																																								
1	0	0	1																																																								
1	1	×	0																																																								
JK 	<table> <tr> <th>J</th><th>K</th><th>Q</th><th>Q_{next}</th></tr> <tr><td>0</td><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>1</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>1</td><td>0</td></tr> </table>	J	K	Q	Q_{next}	0	0	0	0	0	0	1	1	0	1	0	0	0	1	1	0	1	0	0	1	1	0	1	1	1	1	0	1	1	1	1	0	 $Q_{next} = J'K'Q + JK' + JKQ'$ $= J'K'Q + JK'Q + JK'Q' + JKQ'$ $= K'Q(J' + J) + JQ'(K' + K)$ $= K'Q + JQ'$	<table> <tr> <th>Q</th><th>Q_{next}</th><th>J</th><th>K</th></tr> <tr><td>0</td><td>0</td><td>0</td><td>×</td></tr> <tr><td>0</td><td>1</td><td>1</td><td>×</td></tr> <tr><td>1</td><td>0</td><td>×</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>×</td><td>0</td></tr> </table>	Q	Q_{next}	J	K	0	0	0	×	0	1	1	×	1	0	×	1	1	1	×	0
J	K	Q	Q_{next}																																																								
0	0	0	0																																																								
0	0	1	1																																																								
0	1	0	0																																																								
0	1	1	0																																																								
1	0	0	1																																																								
1	0	1	1																																																								
1	1	0	1																																																								
1	1	1	0																																																								
Q	Q_{next}	J	K																																																								
0	0	0	×																																																								
0	1	1	×																																																								
1	0	×	1																																																								
1	1	×	0																																																								

Conversion of JK Flip-flop to D Flip-flop

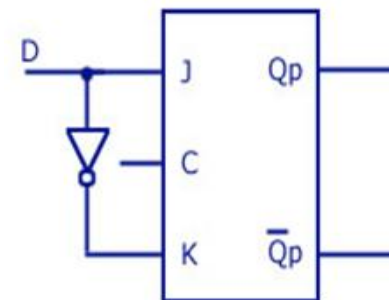
Conversion Table

D Input	Outputs		J-K Inputs	
	Q_p	Q_{p+1}	J	K
0	0	0	0	X
0	1	0	X	1
1	0	1	1	X
1	1	1	X	0

K-maps



Logic Diagram



Conversion of S-R Flip-flop to a JK Flip-flop

Conversion Table

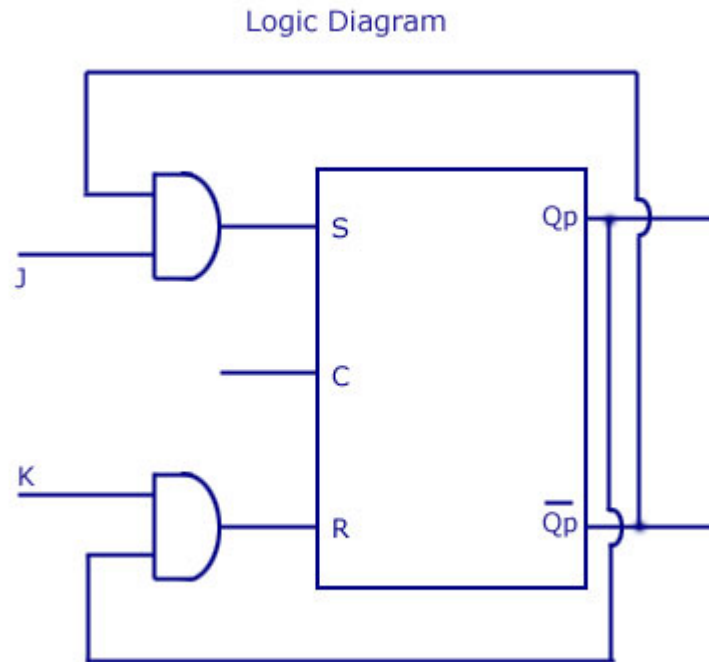
J-K Inputs		Outputs		S-R Inputs	
J	K	Q_p	Q_{p+1}	S	R
0	0	0	0	0	X
0	0	1	1	X	0
0	1	0	0	0	X
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	X	0
1	1	0	1	1	0
1	1	1	0	0	1

J \ KQ_p				
	00	01	11	10
0	0 ⁰	X ¹	0 ³	0 ²
1	1 ⁴	X ⁵	0 ⁷	1 ⁶

J \ KQ_p				
	00	01	11	10
0	X ⁰	0 ¹	1 ³	X ²
1	0 ⁴	0 ⁵	1 ⁷	0 ⁶

$$R = KQ_p$$

Conversion of S-R Flip-flop to a JK Flip-flop



Conversion of D Flip-flop to SR Flip-flop

Conversion Table

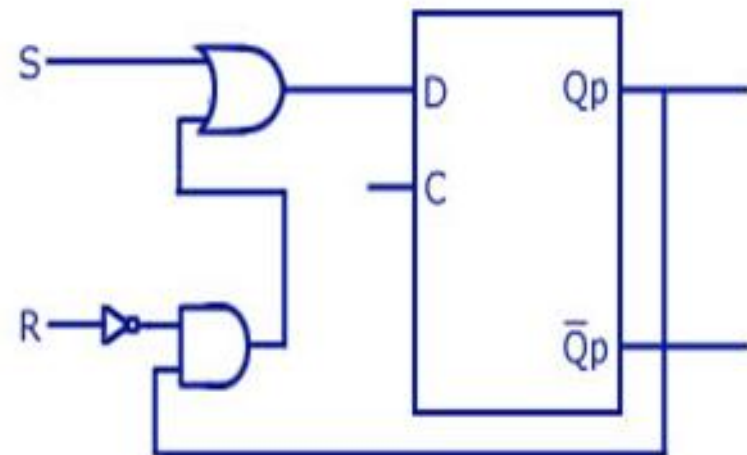
S-R Inputs		Outputs		D Input
S	R	Q _p	Q _{p+1}	
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	Invalid		Dont care
1	1	Invalid		Dont care

K-map

		RQ _p			
		00	01	11	10
S	0	0	1	0	0
	1	1	1	X	X

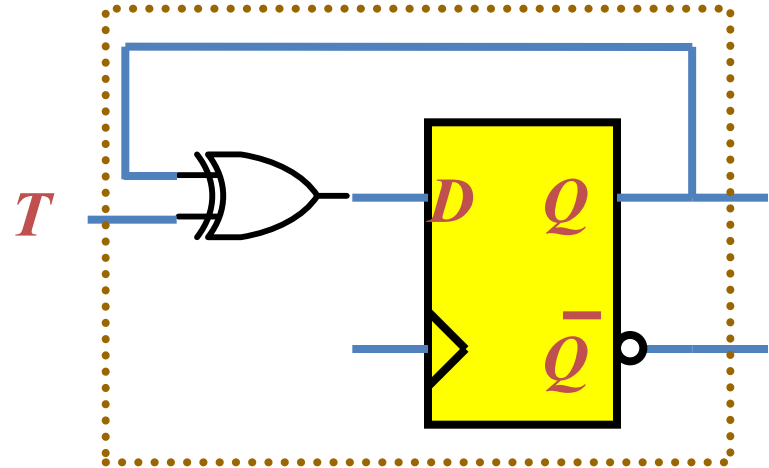
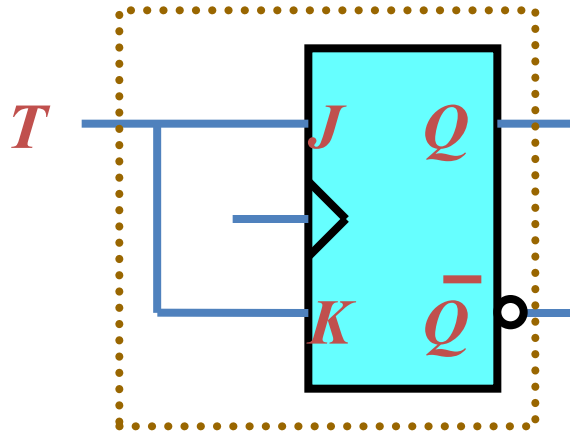
$$D = S + \bar{R}Q_n$$

Logic Diagram



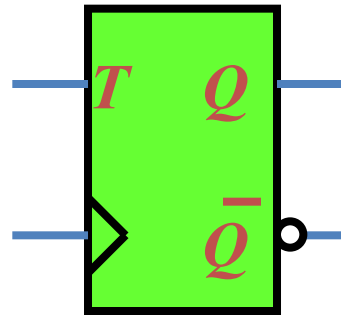
Conversion of D Flip-flop to T Flip-flop

T Flip-Flop

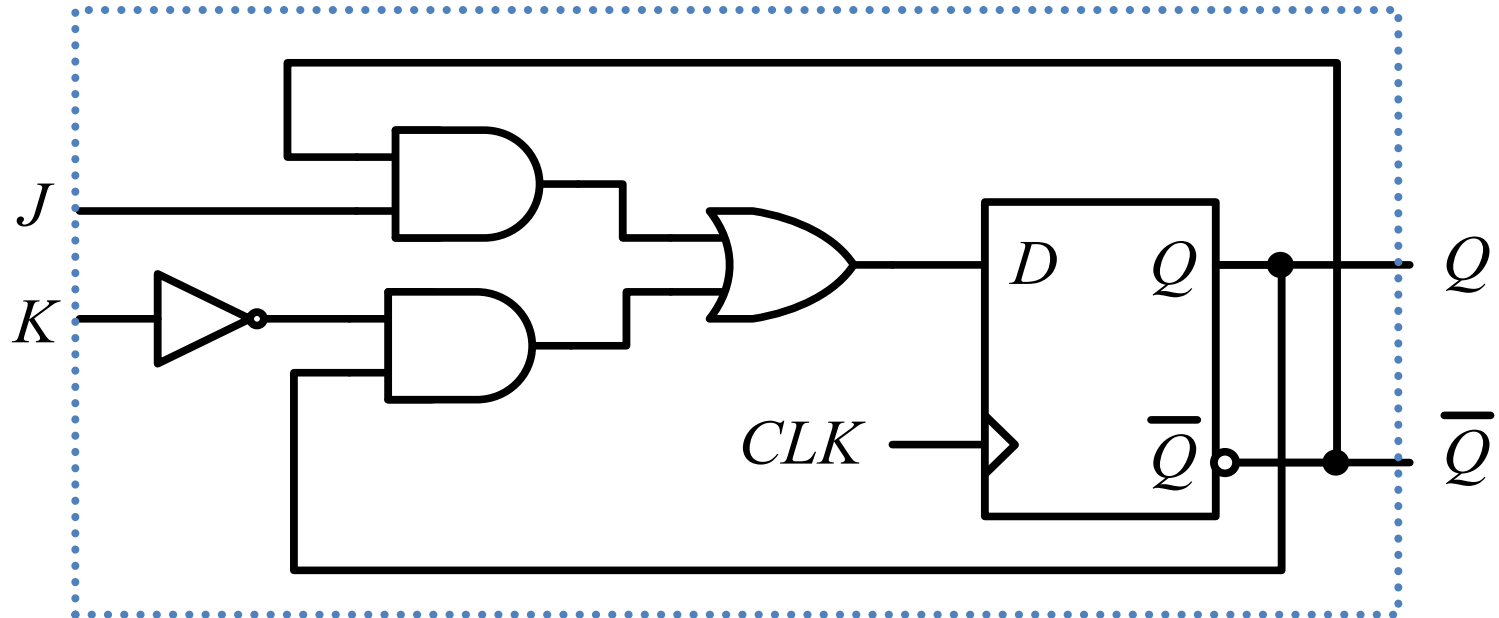


$$D = JQ' + K'Q$$

$$D = TQ' + T'Q = T \oplus Q$$



Conversion of D Flip-flop to a JK Flip-flop



$$D = JQ' + K'Q$$

